Sygnały i Obrazy Cyfrowe

Ćwiczenie nr 1. Skalowanie i obroty

Michał Zywer, nr albumu 290061

28 października 2026

Spis treści

1 Wstęp **2**

2 C3: Pięciokrotne powiększenie obrazu o 10% **3**

3 C4: Pomniejszenie obrazu do rozmiaru oryginalnego **4**

4 C5: Wnioski do C3 oraz C4 **5**

5 C6: Trzykrotne pomniejszenie obrazu o 10% **7**

6 C7: Powiększenie obrazu do rozmiaru oryginalnego **8**

7 C8: Wnioski do C6 oraz C7 **9**

8 C9: Obrócenie obrazu o 360° z krokiem 12° **11**

9 C10: Wnioski do C9 **12**

10 C11, C12, C13: Wnioski końcowe **13**

Spis rysunków

1 Oryginalny obraz testowy. 2
2 C3. Pięciokrotne powiększenie obrazu o 10% algorytmem Okno 3
3 C3. Pięciokrotne powiększenie obrazu o 10% algorytmem Namiot 3
4 C3. Pięciokrotne powiększenie obrazu o 10% algorytmem Keys'a 4
5 C4. Przywrócenie obrazu do oryginalnego rozmiaru algorytmem Okno 4
6 C4. Przywrócenie obrazu do oryginalnego rozmiaru algorytmem Namiot 5
7 C4. Przywrócenie obrazu do oryginalnego rozmiaru algorytmem Keys'a 5
8 C6. Trzykrotne pomniejszenie obrazu o 10% algorytmem Okno 7
9 C6. Trzykrotne pomniejszenie obrazu o 10% algorytmem Namiot 7

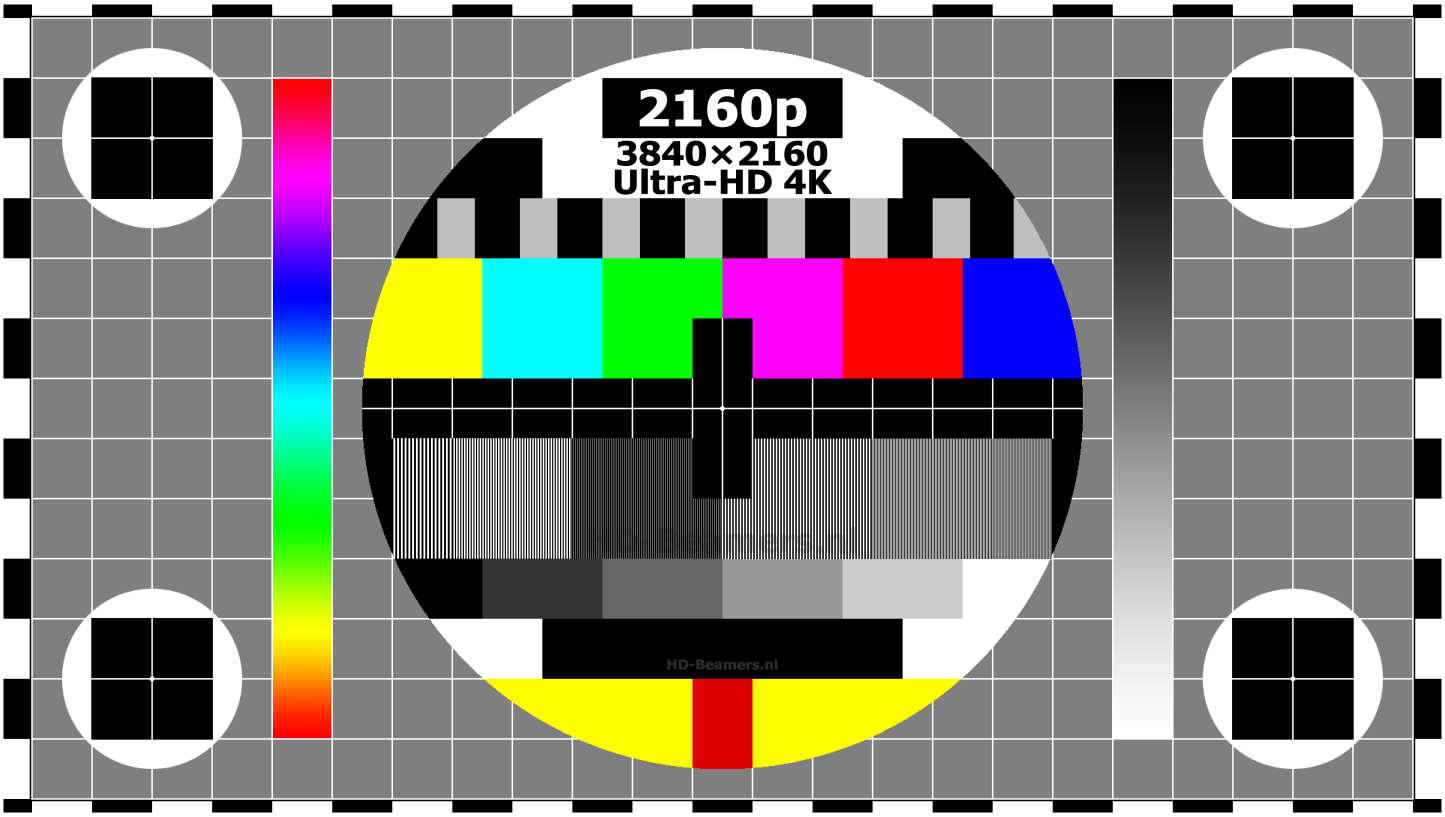
10 C6. Trzykrotne pomniejszenie obrazu o 10% algorytmem Keys'a 8
11 C7. Przywrócenie obrazu do oryginalnego rozmiaru algorytmem Okno 8
12 C7. Przywrócenie obrazu do oryginalnego rozmiaru algorytmem Namiot 9
13 C7. Przywrócenie obrazu do oryginalnego rozmiaru algorytmem Keys'a 9
14 C9. Obraz po obróceniu o 360 stopni algorytmem Okno 11
15 C9. Obraz po obróceniu o 360 stopni algorytmem Namiot 11
16 C9. Obraz po obróceniu o 360 stopni algorytmem Keys'a 12

1 Wstęp

Do napisania tego sprawozdania użyto narzędzi AI.

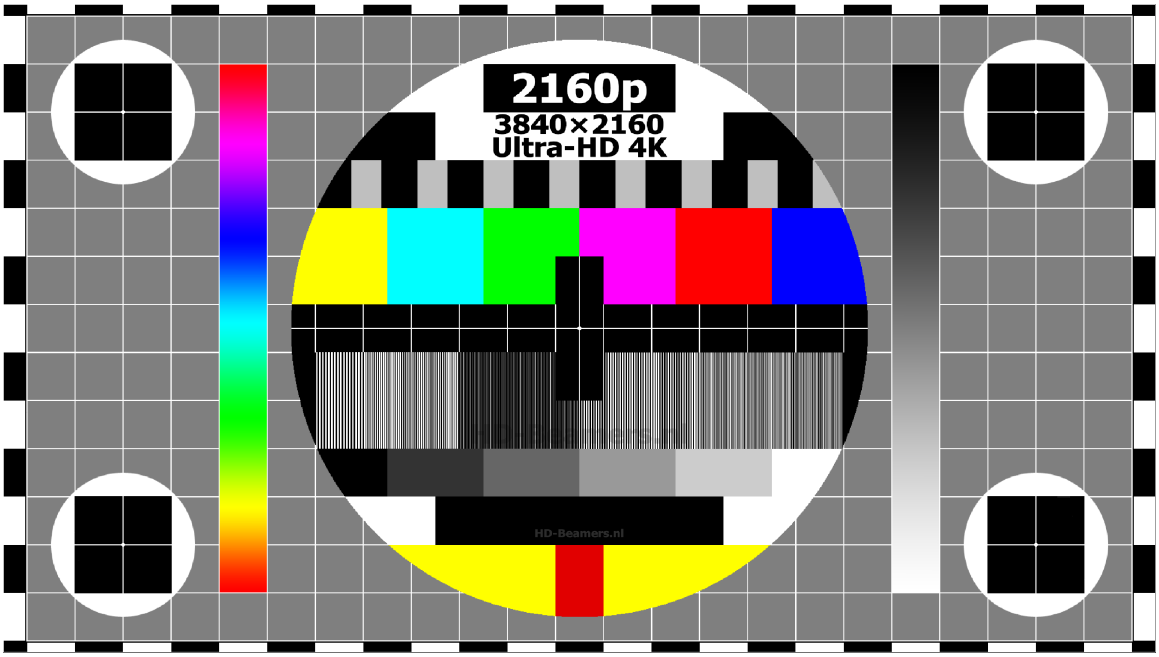
Kod źródłowy programu użytego do skalowania oraz obracania obrazu można znaleźć na mojej stronie Github: `https://github.com/n0rmii/SIOC/tree/main/L02`. Jest on napisany w języku C, a do renderowania używa biblioteki SDL2. Zaimplementowane zostały algorytmy interpolacji typu "Najbliższy sąsiad"(tzw. "Okno"), liniowy (tzw. "Namiot") oraz sześcienny interpolacyjny R. Keys'a (`https://ncorr.com/download/publications/keysbicubic.pdf`). Do jego napisania nie użyto żadnych narzędzi AI.

Wszystkie transformacje do zadań zostały wykonane na poniższym obrazie testowym, pochodzącym ze strony HD-Beamers.nl:

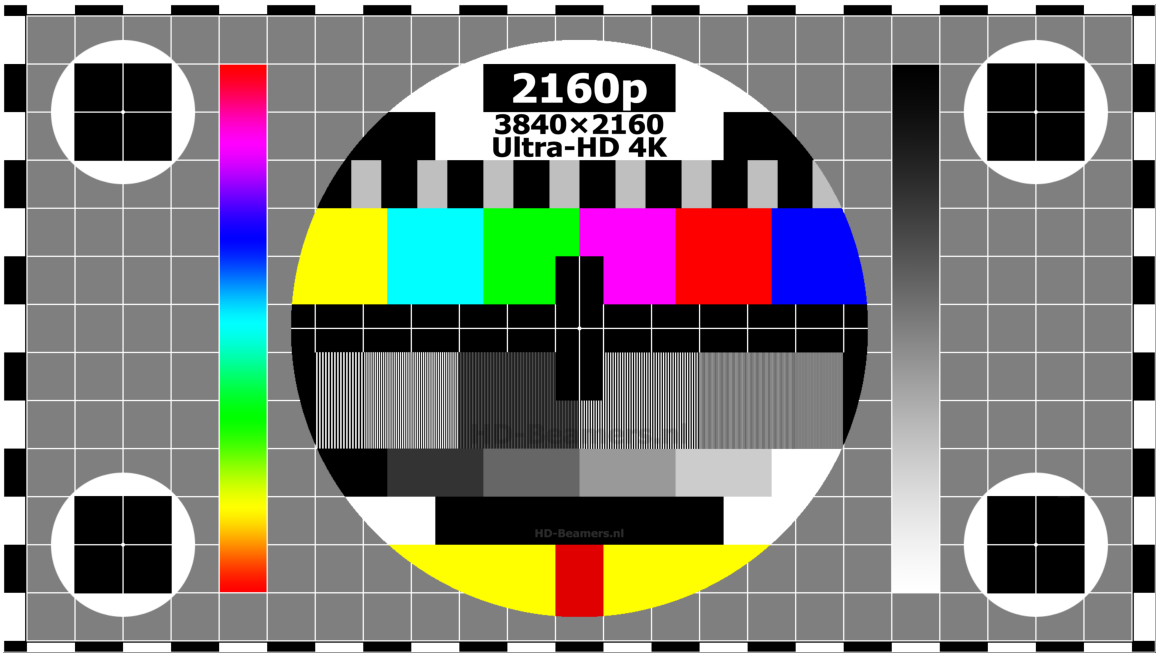


Rysunek 1: Oryginalny obraz testowy.

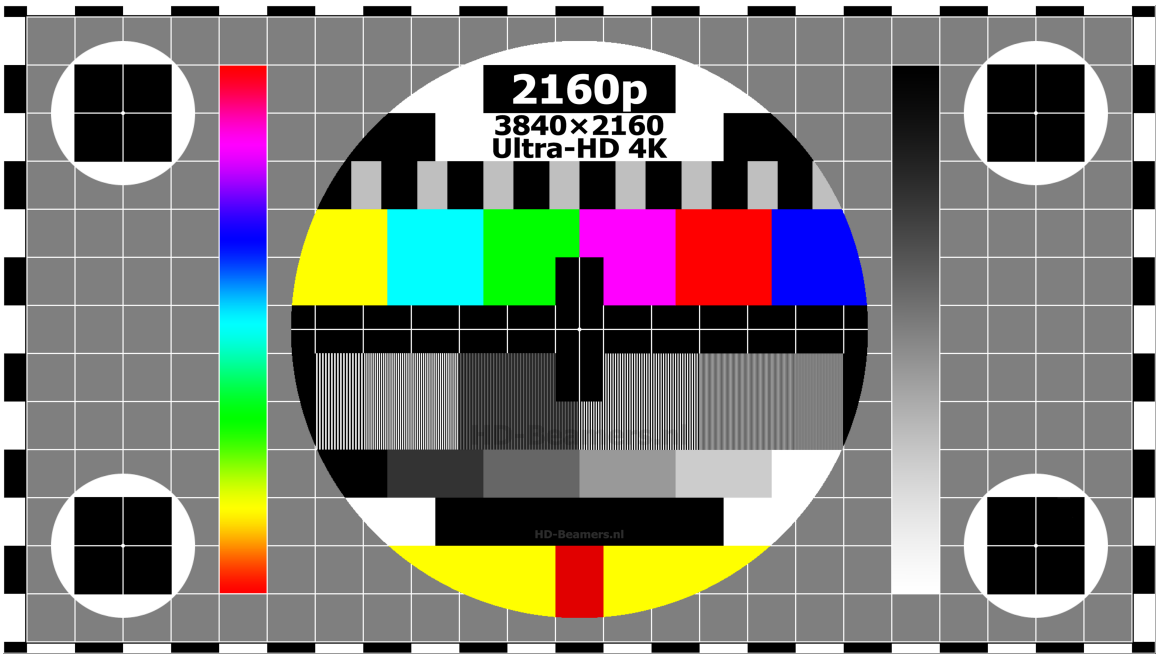
2 C3: Pięciokrotne powiększenie obrazu o 10%



Rysunek 2: C3. Pięciokrotne powiększenie obrazu o 10% algorytmem Okno

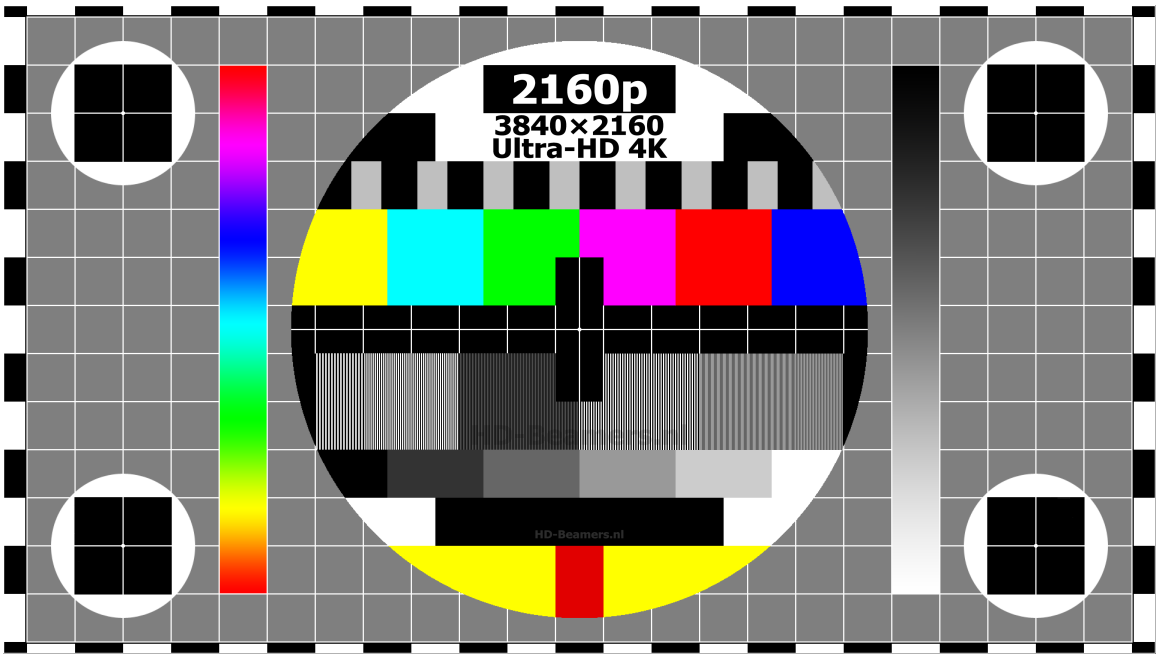


Rysunek 3: C3. Pięciokrotne powiększenie obrazu o 10% algorytmem Namiot

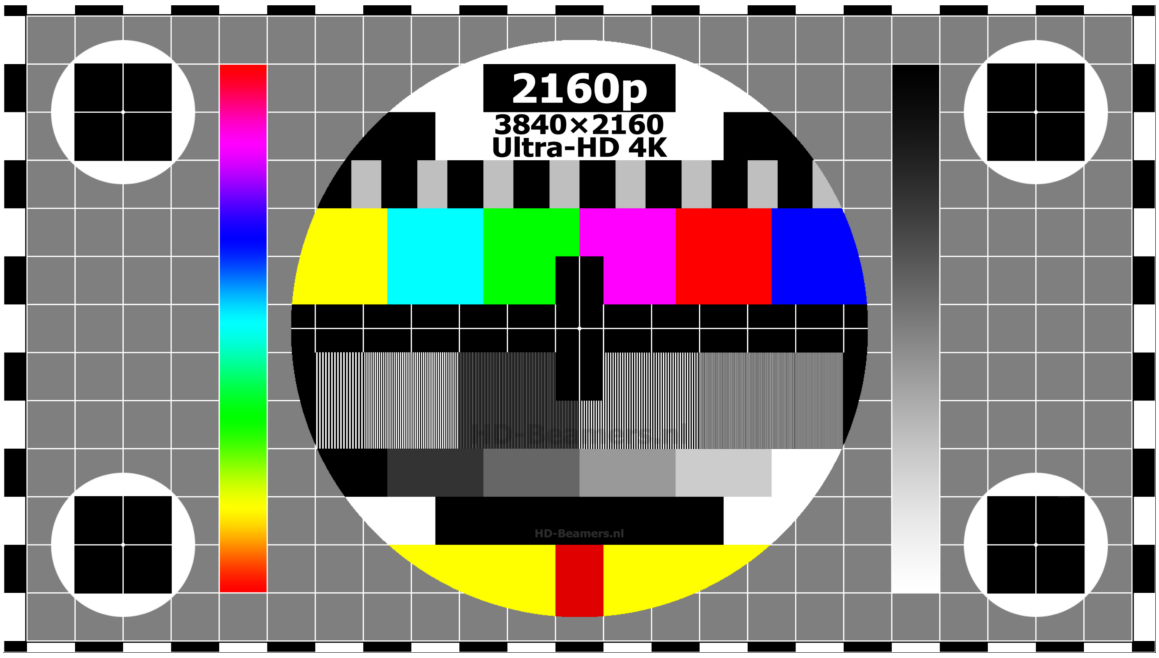


Rysunek 4: C3. Pięciokrotne powiększenie obrazu o 10% algorytmem Keys'a

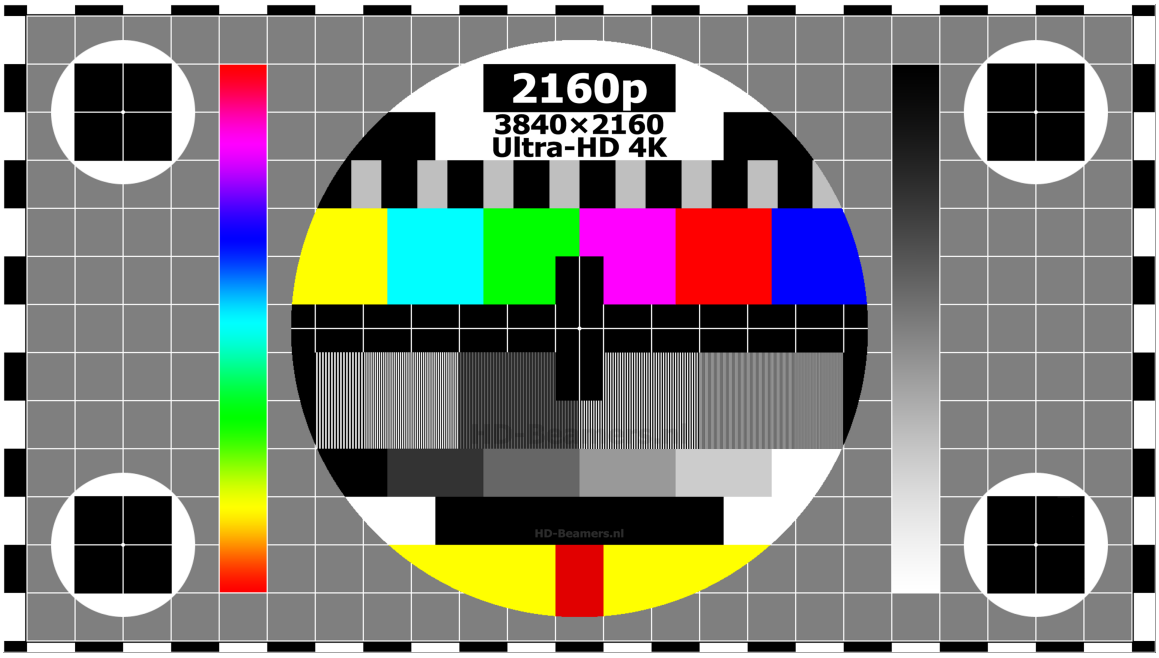
3 C4: Pomniejszenie obrazu do rozmiaru oryginalnego



Rysunek 5: C4. Przywrócenie obrazu do oryginalnego rozmiaru algorytmem Okno



Rysunek 6: C4. Przywrócenie obrazu do oryginalnego rozmiaru algorytmem Namiot



Rysunek 7: C4. Przywrócenie obrazu do oryginalnego rozmiaru algorytmem Keys'a

4 C5: Wnioski do C3 oraz C4

Po pięciu krokach powiększenia o 10% obraz ma rozmiar 6184×3479 pikseli, co odpowiada współczynnikowi $1{,}1^5 \approx 1{,}61$. Następne pomniejszenie (C4) o współczynnik $1{,}1^{-5}$ przywraca rozmiar 3840×2160, więc obraz

wynikowy można porównać piksel po pikselu z obrazem wejściowym (rys. 1).

Powiększanie (C3). Algorytm Okno (rys. 2) nie tworzy nowych wartości, tylko powiela istniejące piksele. Krawędzie zostają ostre, ale widać na nich schodki. Najlepiej to widać na napisach i na okręgu. W polu z pionowymi prążkami (klin rozdzielczości) prążki o tej samej szerokości w oryginale mają po powiększeniu różne szerokości: część jest powielona dwukrotnie, a część nie. Algorytm Namiot (rys. 3) daje obraz bez schodków, ale wyraźnie rozmyty: krawędzie liter są miękkie, a najdrobniejsze prążki tracą kontrast. Algorytm Keys'a (rys. 4) daje obraz gładki i jednocześnie ostrzejszy niż Namiot. Krawędzie napisów i prążki zachowują większy kontrast, ponieważ ujemne „płaty" funkcji Keys'a podbijają ostre przejścia jasności.

Powrót do rozmiaru oryginalnego (C4). W tabeli 1 zestawiono miary błędu obrazów z rys. 5, 6 i 7 względem oryginału: średni błąd bezwzględny (MAE), błąd średniokwadratowy (RMSE), szczytowy stosunek sygnału do szumu (PSNR) oraz odsetek pikseli różniących się od oryginału. Jasność pikseli liczona jest w skali 0–255.

Algorytm	**MAE**	**RMSE**	**PSNR [dB]**	**Zmienione piksele**
Okno	0,18	6,86	31,41	0,1%
Namiot	7,19	24,22	20,45	17,4%
Keys	4,61	17,87	23,09	13,5%

Tabela 1: C4. Porównanie obrazów po operacjach C3 i C4 z obrazem oryginalnym

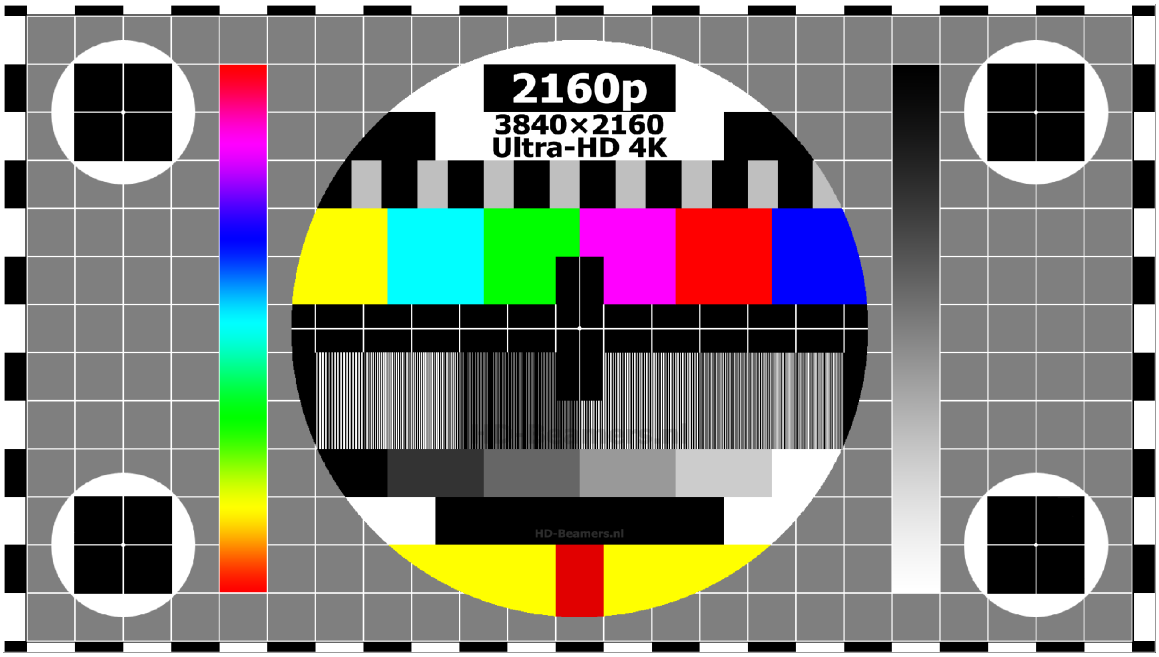
We wszystkich trzech przypadkach jednolite pola (białe, szare, kolorowe) mają po C3 i C4 dokładnie te same wartości co w oryginale, a średnia jasność obrazu się nie zmienia. Różnice dotyczą wyłącznie krawędzi i drobnych szczegółów.

Według miar najlepiej wypada algorytm Okno. Obraz z rys. 5 różni się od oryginału tylko w ok. 0,1% pikseli. Wynika to stąd, że przy powiększaniu każdy piksel oryginału zostaje powielony co najmniej raz, a przy pomniejszaniu algorytm wybiera z powrotem (prawie zawsze) jedną z jego kopii. Złożenie tych operacji jest więc niemal odwracalne. Nie oznacza to jednak, że Okno jest najlepszym algorytmem powiększania: obraz pośredni z rys. 2 ma wyraźne schodki i nierówne prążki.

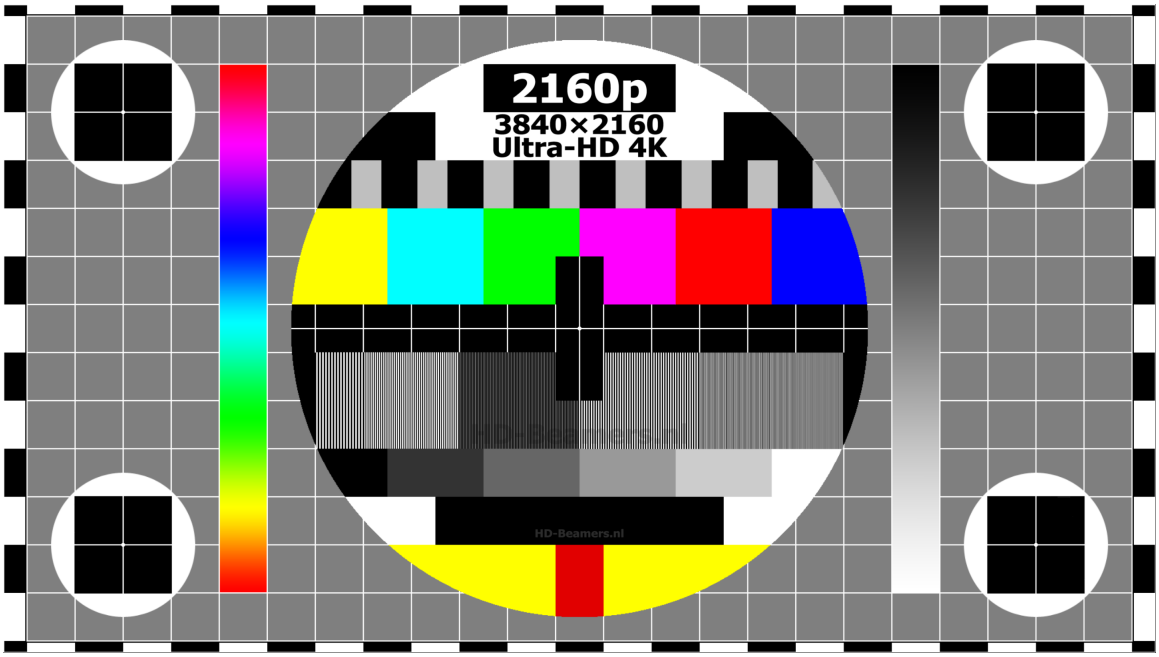
Algorytmy Namiot i Keys'a przy każdej interpolacji uśredniają sąsiednie piksele, więc po dziesięciu kolejnych interpolacjach (5 w górę i 5 w dół) obraz jest trwale zmiękczony. Na rys. 6 widać to szczególnie w klinie rozdzielczości, gdzie najdrobniejsze prążki straciły kontrast i pojawiła się w nich mora, a krawędzie napisów są rozmyte. Na rys. 7 prążki są wyraźniejsze, a napisy ostrzejsze. Odpowiada to lepszym wynikom we wszystkich miarach: zmienionych pikseli jest mniej (13,5% wobec 17,4%), a PSNR jest wyższy o ok. 2,6 dB (23,09 dB wobec 20,45 dB).

Podsumowując: do powiększania najlepszy jest **algorytm Keys'a**. Daje obraz gładki, bez schodków, i najlepiej z trzech zachowuje ostrość krawędzi oraz drobnych szczegółów. Bardzo dobry wynik algorytmu Okno w C4 wynika z odwracalności tej konkretnej sekwencji operacji, a nie z jakości obrazu powiększonego.

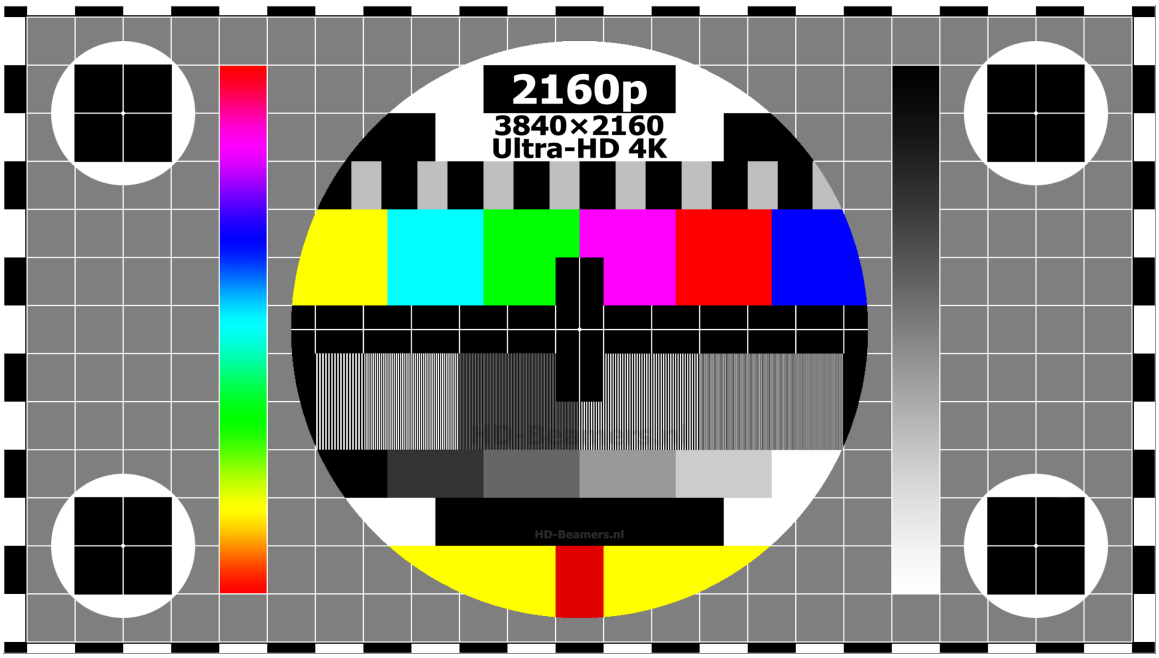
5 C6: Trzykrotne pomniejszenie obrazu o 10%



Rysunek 8: C6. Trzykrotne pomniejszenie obrazu o 10% algorytmem Okno

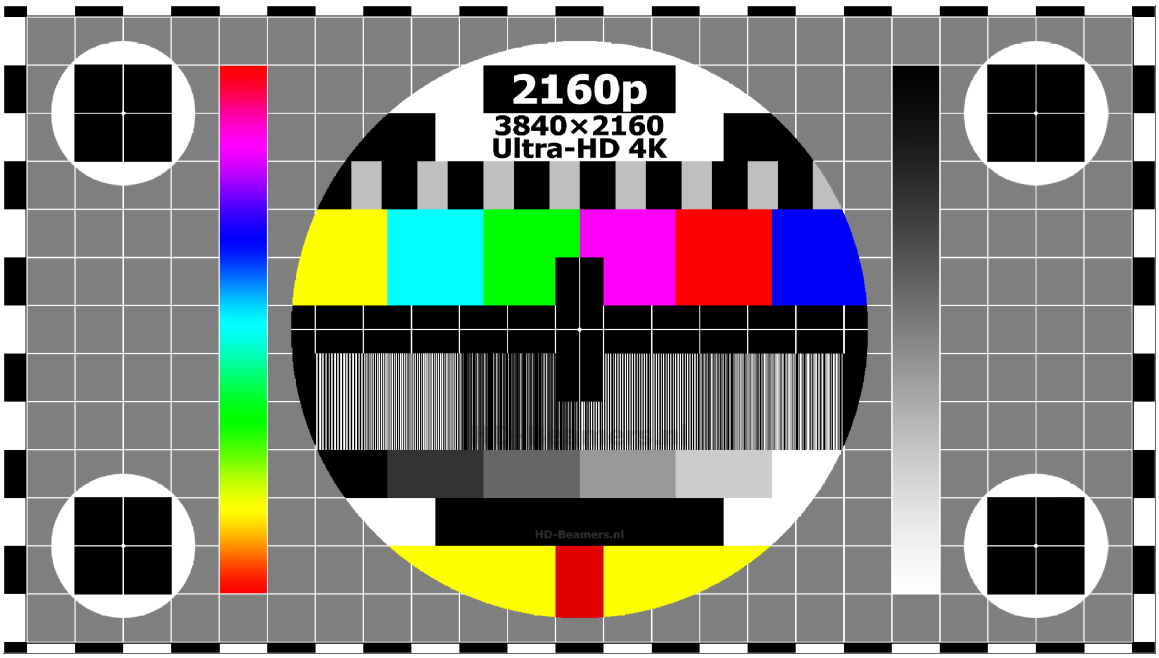


Rysunek 9: C6. Trzykrotne pomniejszenie obrazu o 10% algorytmem Namiot

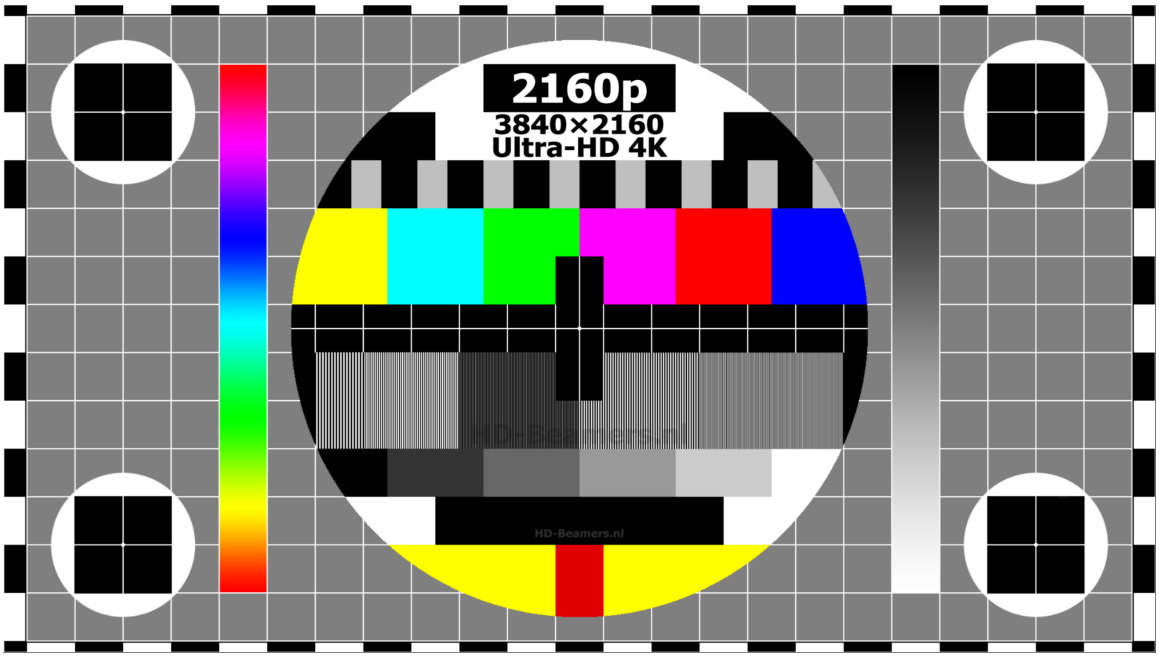


Rysunek 10: C6. Trzykrotne pomniejszenie obrazu o 10% algorytmem Keys'a

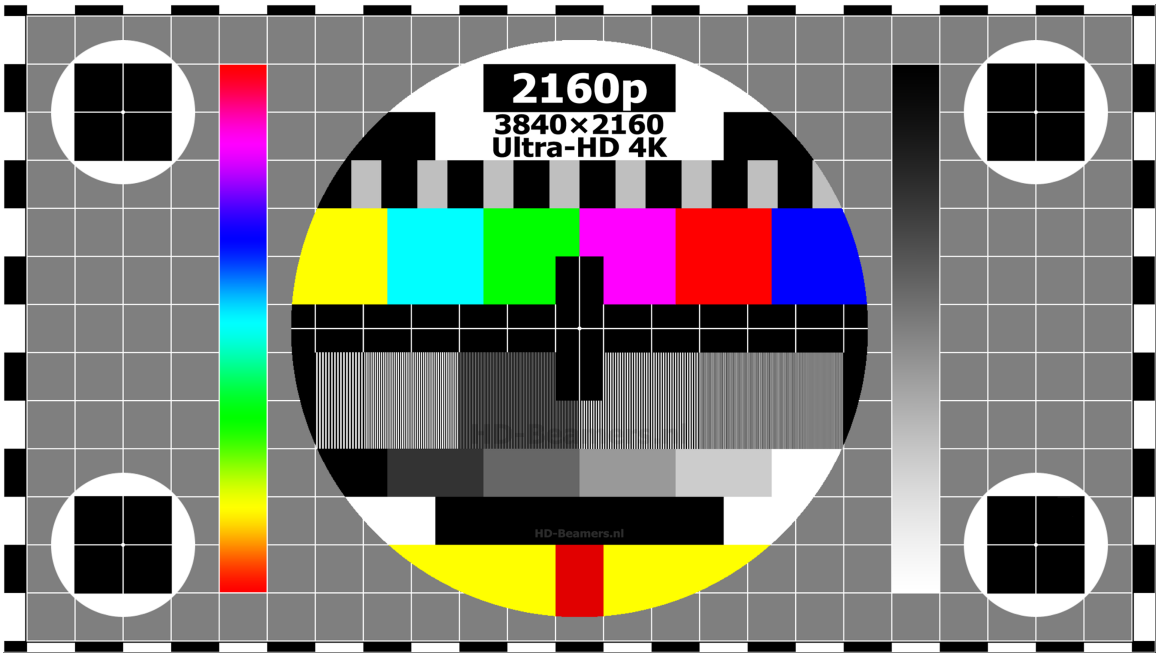
6 C7: Powiększenie obrazu do rozmiaru oryginalnego



Rysunek 11: C7. Przywrócenie obrazu do oryginalnego rozmiaru algorytmem Okno



Rysunek 12: C7. Przywrócenie obrazu do oryginalnego rozmiaru algorytmem Namiot



Rysunek 13: C7. Przywrócenie obrazu do oryginalnego rozmiaru algorytmem Keys'a

7 C8: Wnioski do C6 oraz C7

Po trzech krokach pomniejszenia o 10% obraz ma rozmiar 2885×1623 pikseli (współczynnik $0{,}9^3 = 0{,}729$). Następnie został powiększony z powrotem do 3840×2160. Uwaga: $0{,}9^{-3} \approx 1{,}372$, a nie $1{,}1^3 = 1{,}331$, więc

do powrotu do rozmiaru oryginalnego trzeba użyć odwrotności współczynnika pomniejszenia.

Pomniejszanie (C6). Przy pomniejszaniu liczba próbek maleje, a jeśli obraz zawiera szczegóły o częstotliwości wyższej niż połowa nowej częstotliwości próbkowania (warunek Nyquista), pojawia się aliasing. Obraz testowy jest pod tym względem wymagający, bo zawiera klin rozdzielczości z prążkami szerokości jednego piksela. Dla algorytmu Okno (rys. 8) prążki są nieregularne: części z nich brakuje, a inne zlewają się w szersze pasy. Powstaje fałszywy wzór o niższej częstotliwości, co jest typowym objawem aliasingu. Także napisy i ukośne krawędzie mają schodki. Algorytmy Namiot (rys. 9) i Keys'a (rys. 10) dają bardziej regularne prążki i gładsze krawędzie, ale w najdrobniejszych prążkach również widać mory. Ich funkcje interpolujące mają stałą szerokość (1 lub 2 piksele obrazu wejściowego), niezależnie od współczynnika skali, więc nie usuwają składowych powyżej nowej częstotliwości Nyquista.

Powrót do rozmiaru oryginalnego (C7). Wyniki porównania z oryginałem zestawiono w tabeli 2.

Algorytm	**MAE**	**RMSE**	**PSNR [dB]**	**Zmienione piksele**
Okno	2,90	26,00	19,83	2,3%
Namiot	7,97	26,03	19,82	18,0%
Keys	5,85	21,59	21,44	16,3%

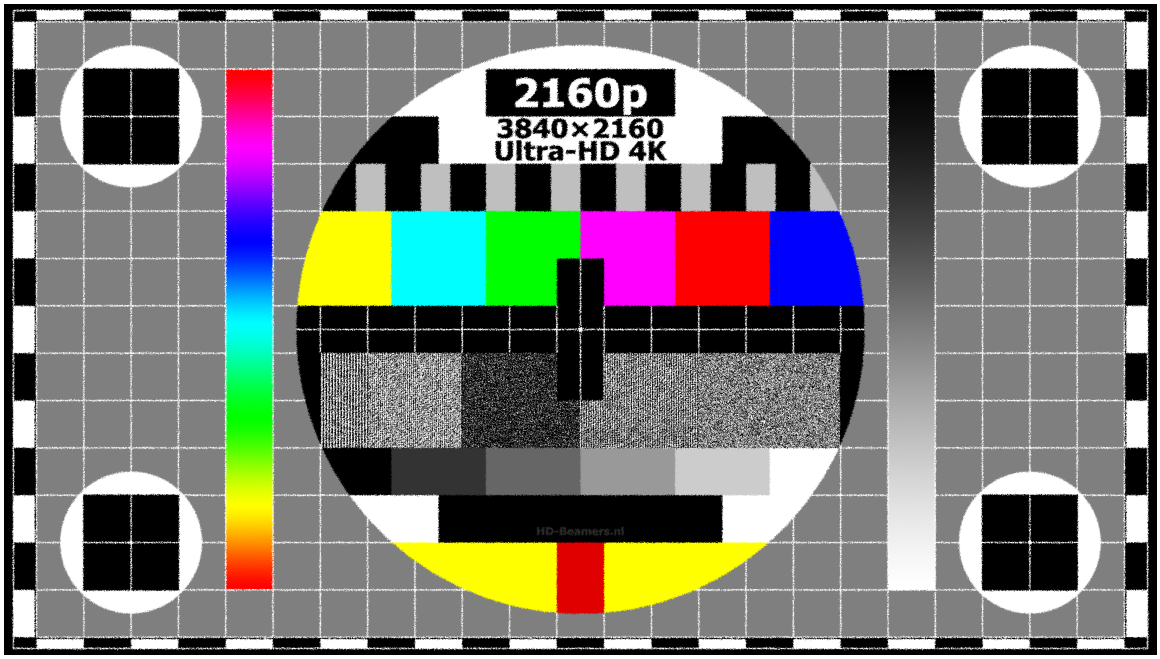
Tabela 2: C7. Porównanie obrazów po operacjach C6 i C7 z obrazem oryginalnym

W odróżnieniu od C3–C4 ta sekwencja nie jest odwracalna dla żadnego z algorytmów. Informacja usunięta przy pomniejszaniu przepada i późniejsze powiększenie jej nie odtworzy. Wszystkie trzy algorytmy mają przy tym niższy PSNR niż w C4 (tab. 1), mimo że interpolacji było mniej (3 w dół i 3 w górę zamiast 5 i 5). Pokazuje to, że utrata informacji przy pomniejszaniu jest groźniejsza niż samo rozmycie.

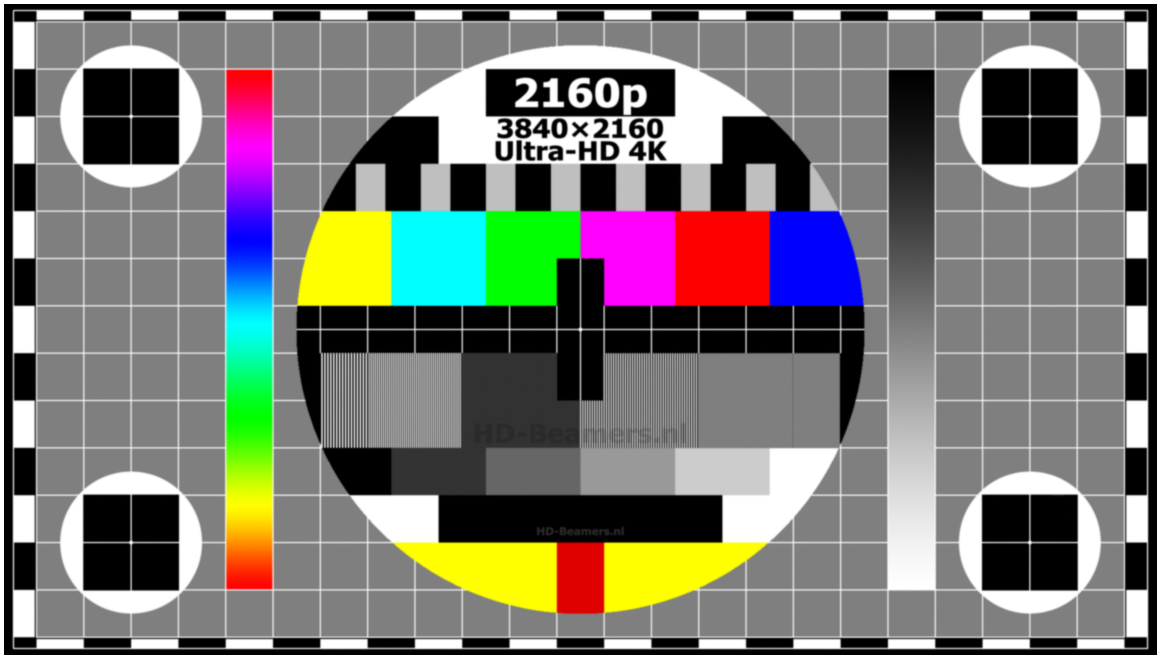
Dla algorytmu Okno (rys. 11) zmieniło się tylko ok. 2,3% pikseli, ale są to zmiany duże i dobrze widoczne: brakujące lub zdublowane linie siatki i prążki, poszarpane krawędzie liter. Dlatego RMSE i PSNR są praktycznie takie same jak dla Namiotu, choć ten zmienił prawie osiem razy więcej pikseli. Obraz z algorytmu Namiot (rys. 12) jest wyraźnie rozmyty, a w najdrobniejszych prążkach zostały ślady mory powstałej przy pomniejszaniu. Algorytm Keys'a (rys. 13) daje najostrzejszy obraz, bez schodków, i ma najlepsze RMSE oraz PSNR (21,44 dB).

Podsumowując: przy pomniejszaniu **najlepszy jest algorytm Keys'a**, a w drugiej kolejności Namiot. Algorytm Okno nie ma żadnego uśredniania, więc najbardziej podatny jest na aliasing. Do pomniejszania warto stosować inne podejście niż do powiększania. Funkcję interpolującą należy rozszerzyć proporcjonalnie do $1/a$ (dla $a < 1$), żeby działała jak filtr dolnoprzepustowy usuwający składowe, których nowa siatka nie może przedstawić. Przy powiększaniu takie filtrowanie nie jest potrzebne, bo nowa siatka jest gęstsza od oryginalnej i aliasing nie występuje.

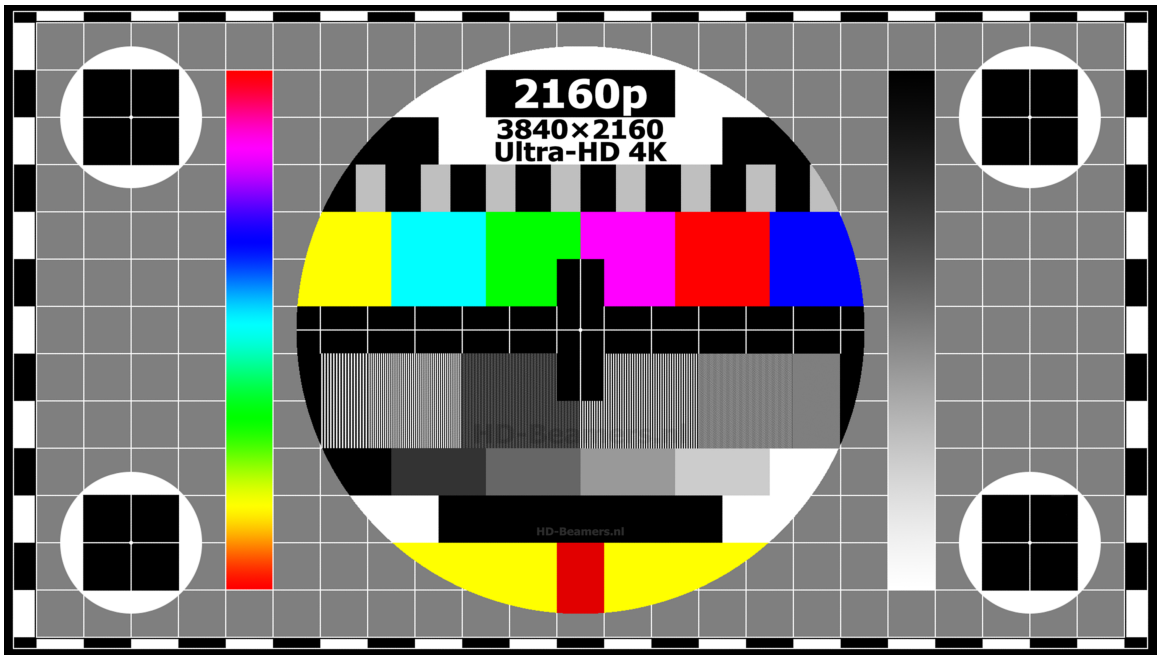
8 C9: Obrócenie obrazu o 360° z krokiem 12°



Rysunek 14: C9. Obraz po obróceniu o 360 stopni algorytmem Okno



Rysunek 15: C9. Obraz po obróceniu o 360 stopni algorytmem Namiot



Rysunek 16: C9. Obraz po obróceniu o 360 stopni algorytmem Keys'a

9 C10: Wnioski do C9

Obraz obrócono 30 razy o 12° względem jego środka, łącznie o 360°. Geometrycznie powinien to być obraz identyczny z oryginałem. Obrazy wynikowe mają płótno 3900×2200 pikseli. Do porównania z oryginałem wycięto z nich środkowy fragment 3840×2160 (przesunięcie o 30 px w poziomie i 20 px w pionie), dla którego błąd był najmniejszy. Wyniki zestawiono w tabeli 3.

Algorytm	**MAE**	**RMSE**	**PSNR [dB]**	**Zmienione piksele**
Okno	10,20	48,17	14,48	7,8%
Namiot	13,06	34,79	17,30	25,6%
Keys	7,28	24,07	20,50	20,0%

Tabela 3: C9. Porównanie obrazów po obrocie o 360° z obrazem oryginalnym

Obrót o kąt, który nie jest wielokrotnością 90°, przenosi węzły siatki pikseli w położenia między węzłami, więc każdy krok wymaga interpolacji. Po 30 krokach błędy kumulują się znacznie bardziej niż przy skalowaniu.

We wszystkich trzech przypadkach jednolite pola (białe, szare, kolorowe) mają po obrocie dokładnie te same wartości co w oryginale, a średnia jasność obrazu praktycznie się nie zmienia (ok. 121,8 wobec 122,1). Błędy dotyczą więc wyłącznie krawędzi i drobnych szczegółów, czyli miejsc, w których interpolacja musi „odgadnąć" wartości między pikselami.

Algorytm Okno (rys. 14) kopiuje istniejące piksele, więc nie tworzy nowych kolorów. Każdy krok przesuwa jednak część pikseli nawet o pół piksela, a po 30 obrotach te przesunięcia sumują się losowo. Wszystkie krawędzie są przez to postrzępione i pokryte szumem typu „sól i pieprz". Napisy są poszarpane, a klin roz-

dzielczości zamienił się w szum. Zmienionych pikseli jest najmniej (7,8%), ale błędy są duże (zamiana bieli na czerń i odwrotnie), dlatego algorytm ma największe RMSE i najmniejszy PSNR.

Algorytm Namiot (rys. 15) daje krawędzie gładkie, ale po 30 kolejnych uśrednieniach obraz jest wyraźnie rozmyty. Napisy są miękkie, a najdrobniejsze prążki klina zlały się w szare pole. Zmienionych pikseli jest najwięcej (25,6%), bo rozmycie obejmuje otoczenie każdej krawędzi. Algorytm Keys'a (rys. 16) zachowuje krawędzie i drobne szczegóły znacznie lepiej. Napisy pozostają ostre, a prążki klina są nadal wyraźnie widoczne, choć z lekką morą w najdrobniejszych z nich. Ma też najlepsze wyniki we wszystkich trzech miarach (MAE 7,28, PSNR 20,50 dB).

Podsumowując: do obracania **najlepszy jest algorytm Keys'a**, zarówno według miar liczbowych, jak i w ocenie wizualnej. Drugi jest Namiot, a najgorsze Okno. Jest to ta sama funkcja interpolacji, którą wybrano do skalowania. Przy obrotach jej przewaga nad algorytmem Okno jest nawet większa, bo algorytm najbliższego sąsiada nie ma tu „odwracalności" obserwowanej w C4.

10 C11, C12, C13: Wnioski końcowe

We wszystkich operacjach algorytm Keys'a dawał obrazy najlepsze wizualnie, algorytm Namiot nieco gorsze (rozmyte), a algorytm Okno najgorsze (schodki, aliasing, szum przy obrotach). Wyjątkiem w miarach liczbowych jest przypadek, w którym algorytm Okno korzystał z odwracalności sekwencji operacji (C4). Pokazuje to, że *jakość* mierzona prostą miarą piksel po pikselu (MAE, RMSE, PSNR) i *wygląd* obrazu oceniany przez człowieka to nie to samo. PSNR mocno karze niewielkie, równomierne przesunięcie jasności, którego oko praktycznie nie zauważa, a nie odróżnia na przykład rozmycia od szumu o tej samej energii, choć wizualnie są to zupełnie różne defekty. Miary liczbowe nadają się do obiektywnego porównania z obrazem wzorcowym, gdy taki istnieje. Gdy obraz ma być oglądany przez człowieka, lepsza jest ocena wizualna lub miary percepcyjne, np. SSIM.

Ważna jest też dokładność samych obliczeń. Sumę ważoną pikseli należy zbierać w zmiennej zmiennoprzecinkowej (`double`), a wynik zaokrąglać (`lround`) i ograniczać do zakresu 0–255 tylko raz, na końcu. Obcinanie wyniku do liczby całkowitej po każdym składniku sumy powodowałoby systematyczne ściemnianie obrazu o ok. 1 poziom jasności na każde przejście interpolacji 1D, które przy wielokrotnych operacjach (np. 30 obrotach w C9) bardzo szybko by się kumulowało. Dzięki takiemu podejściu we wszystkich wynikach jednolite pola mają dokładnie te same wartości co w oryginale, a błędy dotyczą wyłącznie krawędzi i drobnych szczegółów. Przy wielokrotnych operacjach najlepiej przechowywać w postaci zmiennoprzecinkowej również obrazy pośrednie, żeby błąd zaokrąglenia do 8 bitów nie powtarzał się w każdym kroku.

C11: Powiększanie. Zamiast kilku kolejnych powiększeń lepiej wykonać jedno powiększenie od razu o łączny współczynnik $(1{,}1^5)$. Każda dodatkowa interpolacja dodaje rozmycie, a przy Keys'ie także przeregulowania na krawędziach. Można też użyć funkcji interpolującej wyższego rzędu o szerszym nośniku, np. Lanczosa. Rozmycie można częściowo zmniejszyć wyostrzaniem (np. filtrem typu unsharp mask), kosztem wzmocnienia szumu. Nie da się jednak odtworzyć szczegółów, których nie było w obrazie wejściowym, więc poprawiamy wtedy wygląd, a nie wierność.

C12: Pomniejszanie. Kluczowe jest ograniczenie aliasingu przez filtrację dolnoprzepustową przed decymacją albo, co jest równoważne, przez rozszerzenie funkcji interpolującej o współczynnik $1/a$, tak by każdy piksel wynikowy uśredniał wszystkie piksele oryginału, które „pokrywa" (uśrednianie powierzchniowe, ang. *area averaging*, lub przeskalowany Namiot czy Keys). Przy dużym zmniejszeniu można też pomniejszać w kilku etapach co najwyżej dwukrotnie, z filtracją w każdym etapie (piramida obrazów, mipmapy). Tak jak przy powiększaniu, lepiej wykonać jedną operację o łączny współczynnik niż kilka kolejnych.

C13: Obracanie. Największą poprawę daje unikanie wielokrotnej interpolacji. Zamiast obracać obraz już obrócony, należy pamiętać łączny kąt i za każdym razem obracać obraz *oryginalny* o kąt sumaryczny (tu $30 \cdot 12^\circ = 360^\circ$, czyli przekształcenie tożsamościowe bez żadnej utraty jakości). Obroty o wielokrotności 90° warto realizować przez zwykłe przestawienie pikseli, bez interpolacji. Dla pozostałych kątów pomaga interpolacja wyższego rzędu (Keys, Lanczos) oraz nadpróbkowanie (ang. *supersampling*): obrót obrazu wcześniej powiększonego i pomniejszenie wyniku z filtracją. Można też rozłożyć obrót na trzy przekształcenia ścinające 1D (metoda Paetha), w których każda interpolacja jest jednowymiarowa i dobrze uwarunkowana.